

ECE 382N-Sec (FA26):

L9: Memory Encryption and Integrity Protection

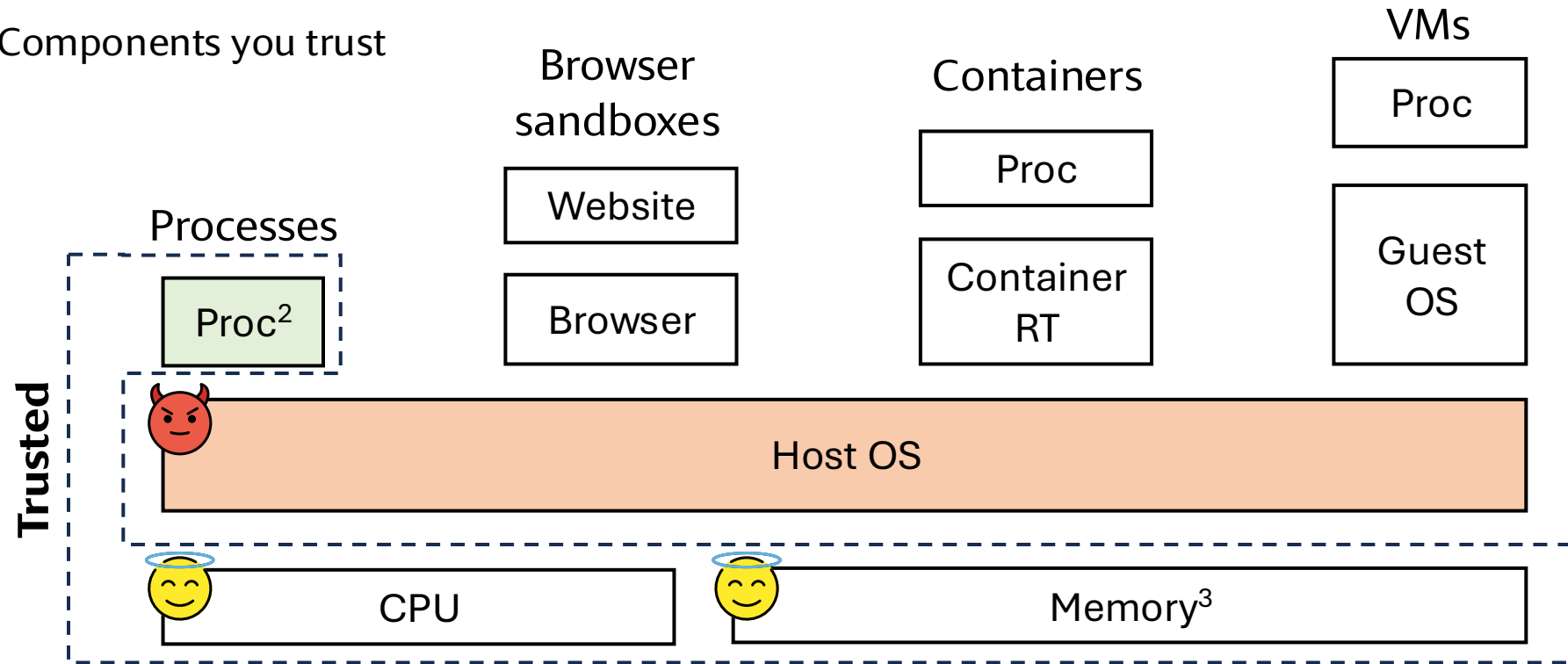
Neil Zhao

neil.zhao@utexas.edu

Trusted-Execution Environments (TEE)¹

 Your program

 Components you trust



¹TEE is a somewhat overloaded term. We focus on hardware-based TEEs

²The process may be divided into trusted and untrusted parts

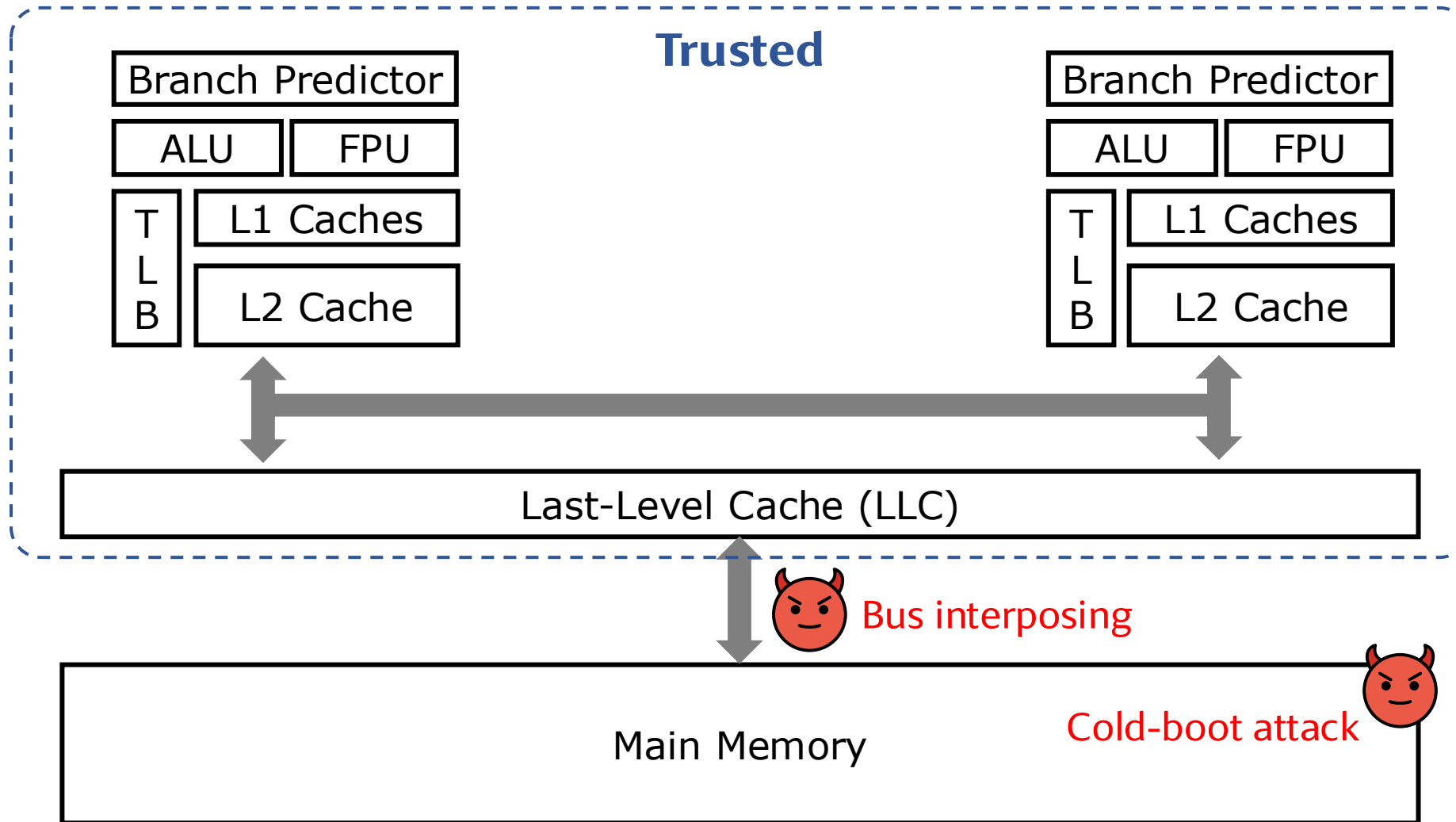
³Depending on the memory type and threat model, it may or may not be trusted

(Common*) Security Goals of TEEs

		Example Attacks	
		Software Attack	Physical Attack
✓	Confidentiality	Attacker cannot directly access my private program states (Side channel? Spectre?)	OS reads my pages Bus snooping
✓	Integrity	Attacker cannot tamper with my program states (Freshness: Program state is up-to-date)	OS writes my pages ? Bus spoofing
✗	Availability	Attacker refuses to execute or give enough resources to my program	OS allocates no CPU time Pull the plug

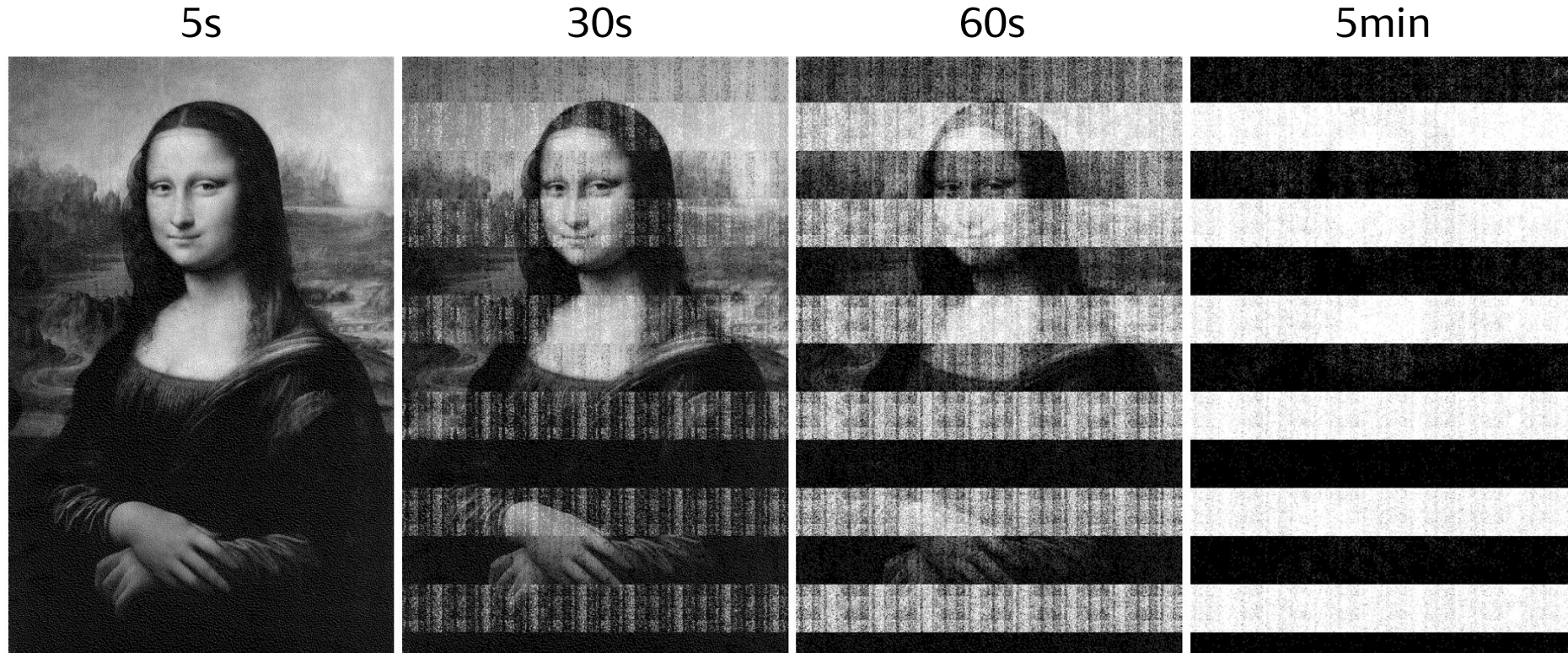
*Many variants exist

The Need for Memory Encryption and Integrity Protection



Cold-Boot Attack

Observation: Data in DRAM cells can survive for seconds after losing power
⇒ The window can be extended by cooling the DRAM to a low temperature



Source: Halderman et al., "Lest We Remember: Cold Boot Attacks on Encryption Keys," USENIX Sec '08

Cold-Boot Attack

Observation: Data in DRAM cells can survive for seconds after losing power
⇒ The window can be extended by cooling the DRAM to a low temperature

	Seconds w/o power	Error % at operating temp.	Error % at -50°C
A	60	41	(no errors)
	300	50	0.000095
B	360	50	(no errors)
	600	50	0.000036
C	120	41	0.00105
	360	42	0.00144
D	40	50	0.025
	80	50	0.18

Table 2: Effect of cooling on error rates

Source: Halderman et al., “Lest We Remember: Cold Boot Attacks on Encryption Keys,” USENIX Sec ’08

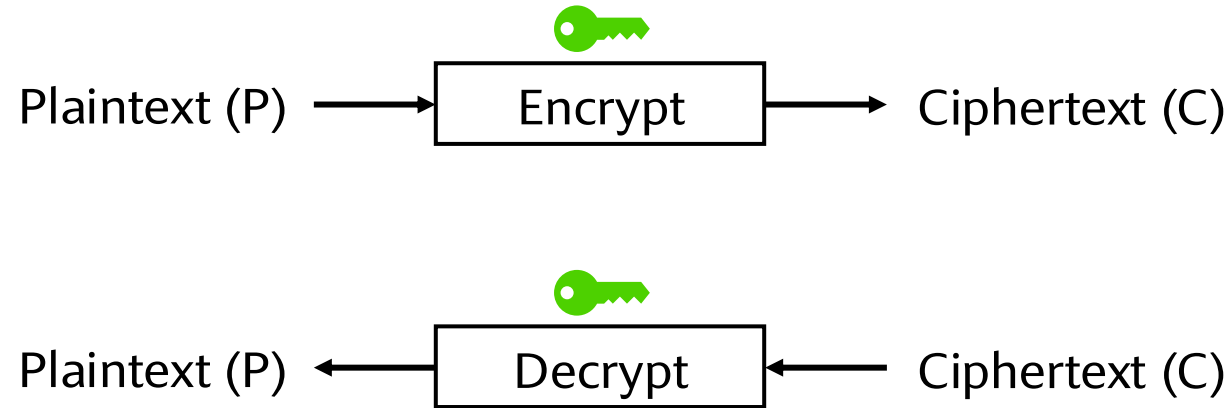
A Typical Attack Process:

- Cool down the DRAM
- Suddenly power off the machine
⇒ Take a “snapshot” of the memory
- Boot into a USB drive that contains the program for dumping the memory

Can dump the entire memory, including the disk encryption key found in the memory

Demo: <https://www.youtube.com/watch?v=XfUIRsE3ymQ>

Hammer 4: Symmetric Cipher



The permutation is determined by the key and informally,
the permutation should look random

Hammer 4: Symmetric Cipher

In general, we want different ciphertexts if we encrypt the same plaintext twice

Disease		Disease (Encrypted)
Flu		C#@husd
Flu		C#@husd
Diabetes	Pad to the same length and encrypt using the same key	yv07*we
Covid		fgh8973
Flu		C#@husd
Covid		fgh8973

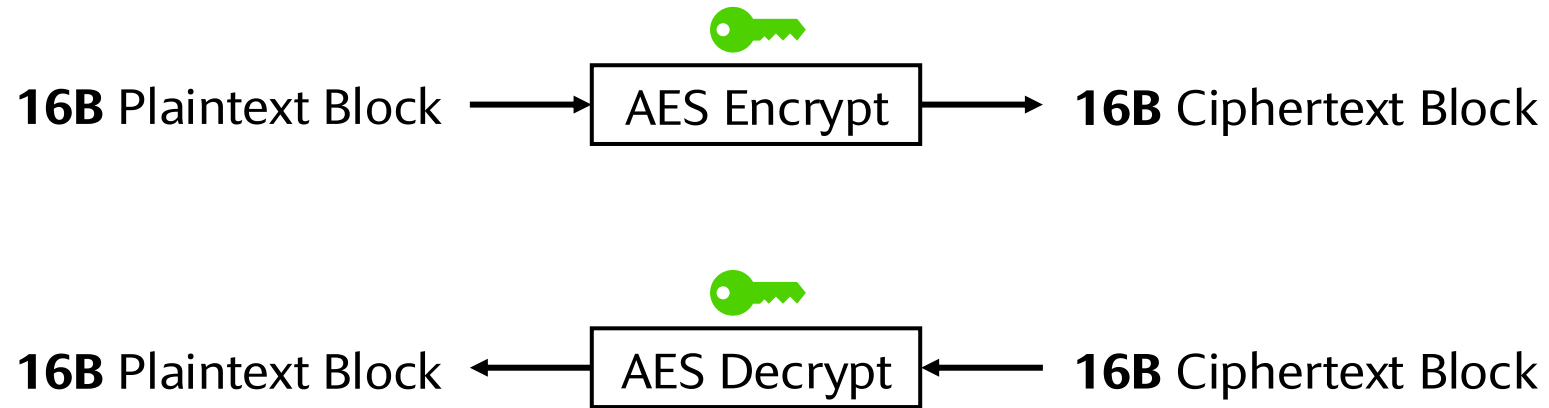
Frequency analysis reveals the disease

One-Time Pad

Plaintext (N bits)	0	1	0	0	0	0	0	1	1 ...
	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$
Key (N random bits)	0	0	1	0	1	1	0	0	1 ...
	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$
Ciphertext (N bits)	0	1	1	0	1	1	0	1	0 ...
	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$
Same Key	0	0	1	0	1	1	0	0	1 ...
	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$
Plaintext (N bits)	0	1	0	0	0	0	0	1	1 ...

Advanced Encryption Standard (AES)

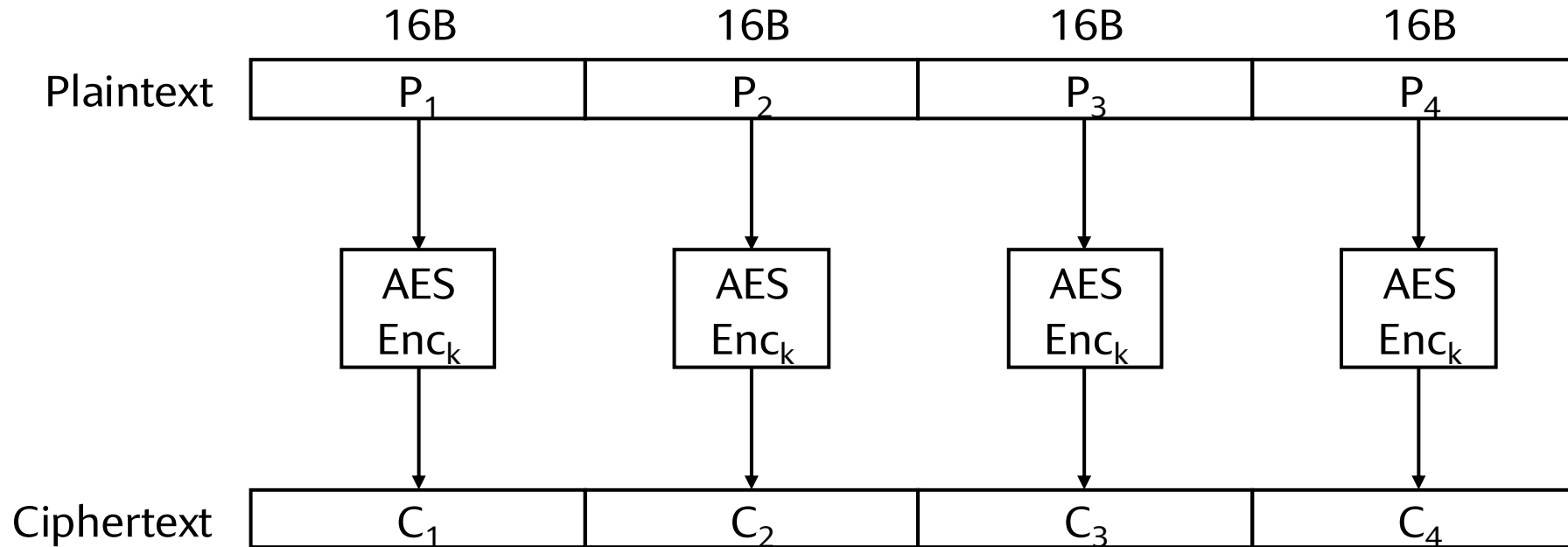
AES is a popular block cipher



Many high-end processors have hardware-accelerated AES instructions

How to use AES to encrypt a message of any length?

Electronic Codebook (ECB) Mode



Same plaintext blocks are encrypted into the same ciphertext block!

Electronic Codebook (ECB) Mode

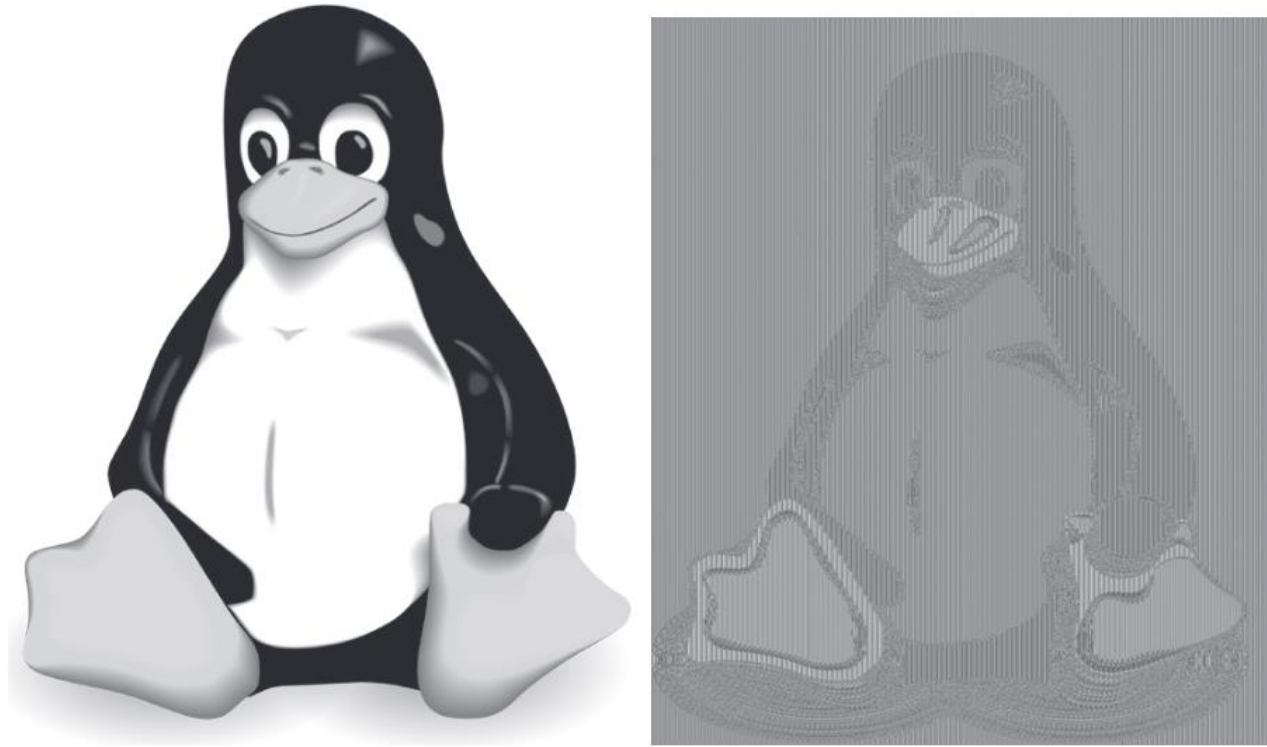
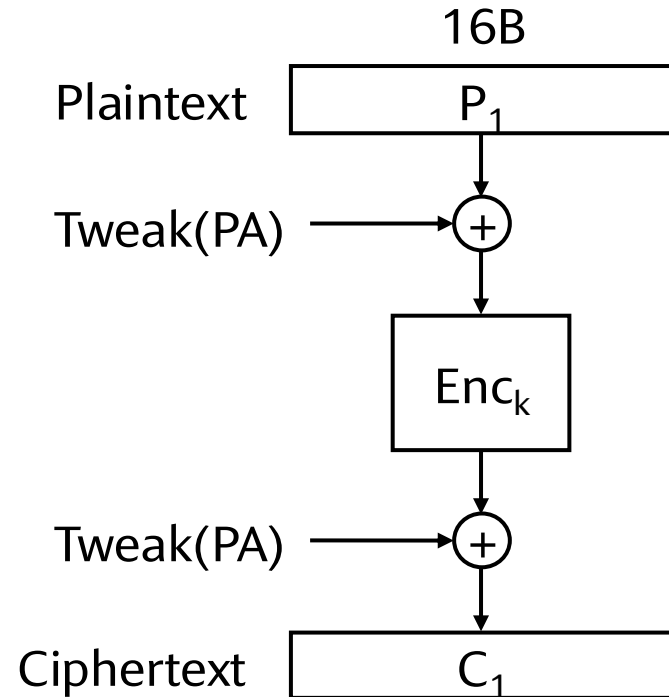


Figure 4-7: The original image (left) and the ECB-encrypted image (right)

Source: "Serious Cryptography: A Practical Introduction to Modern Encryption" by Jean-Philippe Aumasson

XOR-Encrypt-XOR (XEX) Mode

The encryption depends on the physical address (PA) of the data block



$$C = Enc_k(P \oplus Tweak(PA)) \oplus Tweak(PA)$$



Achieve spatial uniqueness---i.e., the same plaintext block at different PAs are encrypted to different ciphertext blocks

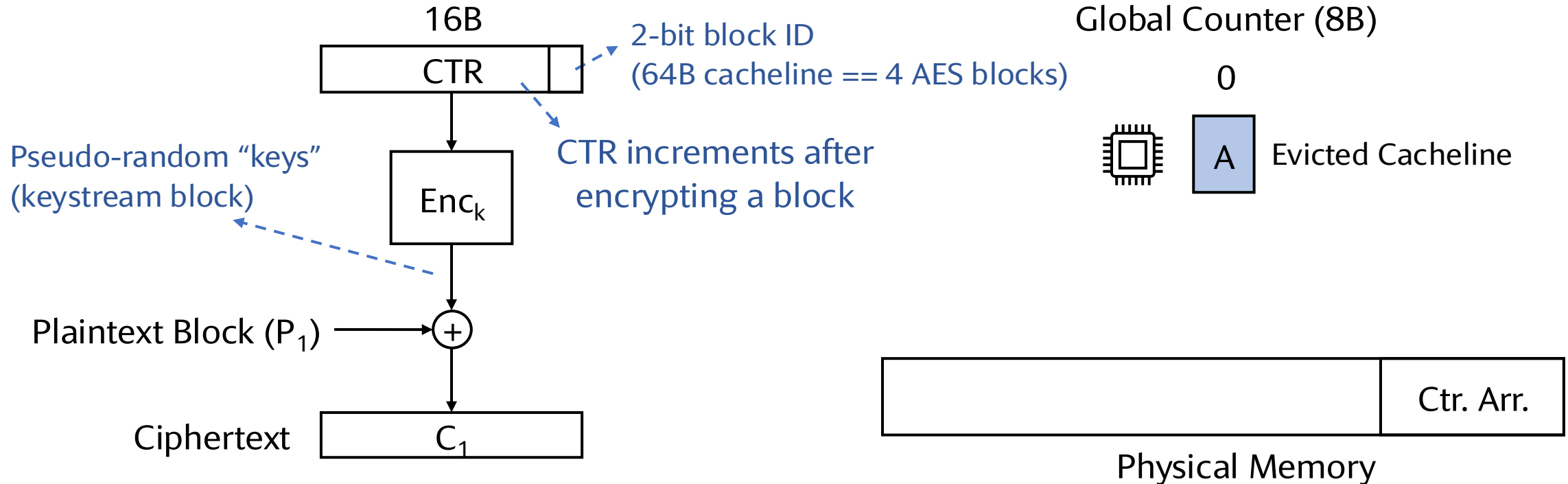


Deterministic encryption at a given location

Attacks on AMD SEV(-SNP):

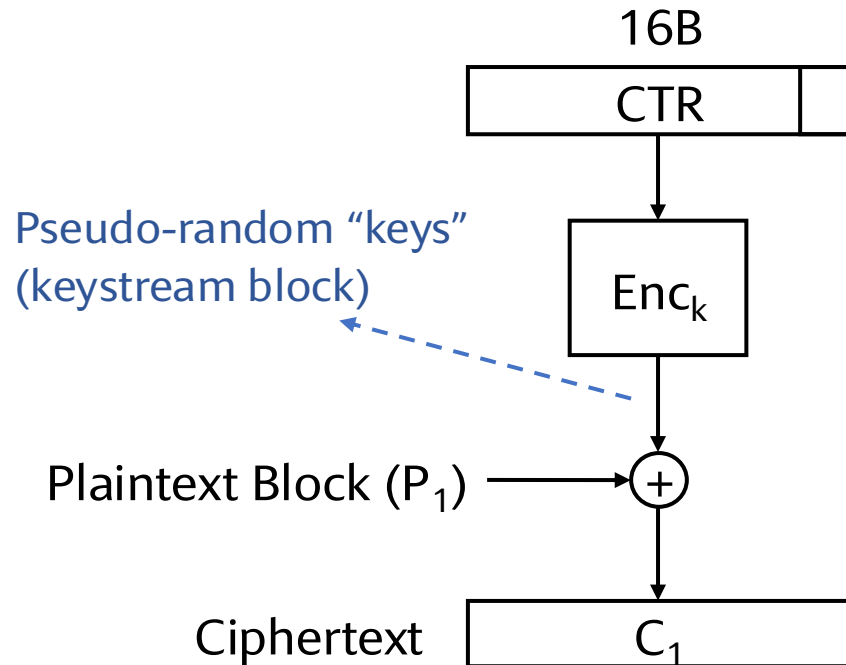
- Li et al, CIPHERLEAKS: Breaking Constant-time Cryptography on AMD SEV via the Ciphertext Side Channel, USENIX'21
- Li et al, A Systematic Look at Ciphertext Side Channels on AMD SEV-SNP, S&P'22

Counter (CTR) Mode

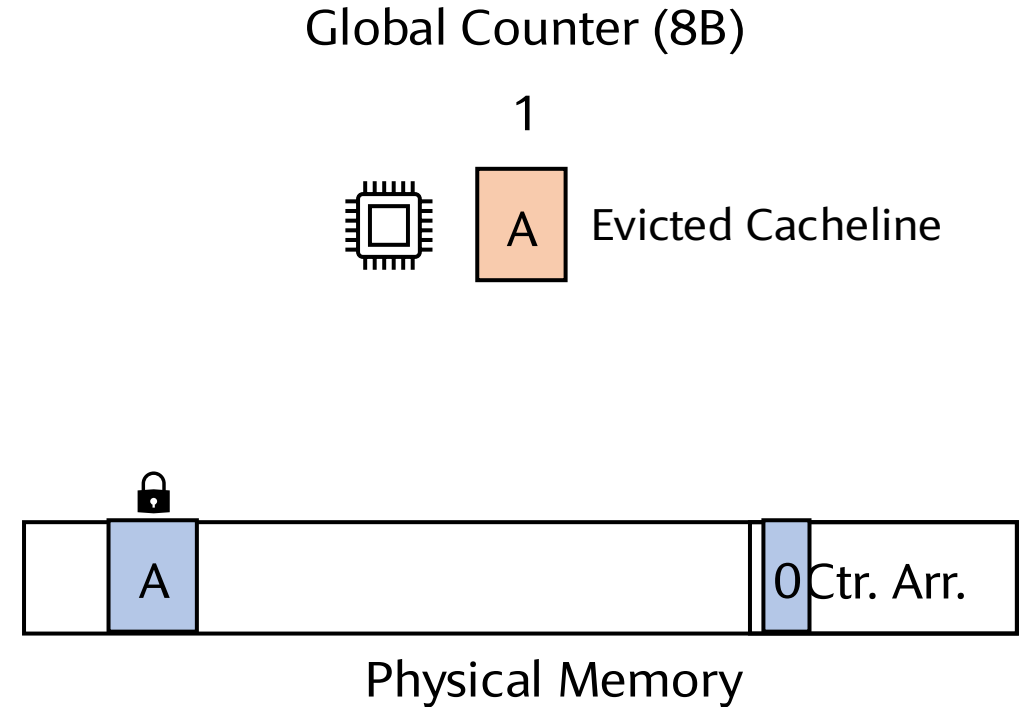


$$C = Enc_k(CTR) \oplus P \quad P = Enc_k(CTR) \oplus C$$

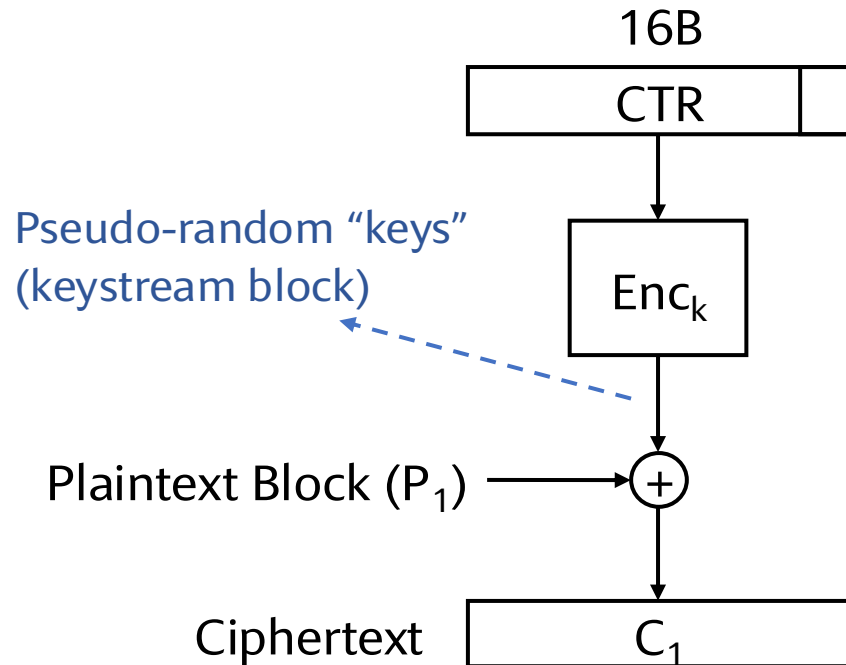
Counter (CTR) Mode



$$C = Enc_k(CTR) \oplus P \quad P = Enc_k(CTR) \oplus C$$



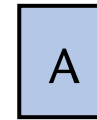
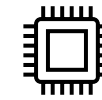
Counter (CTR) Mode



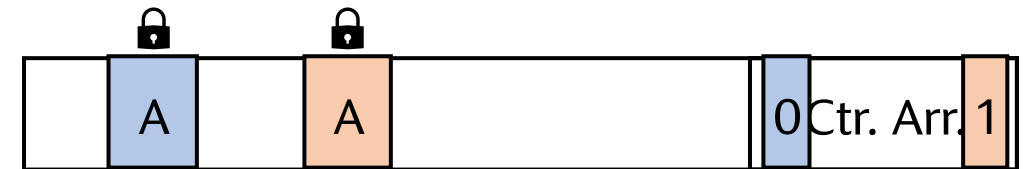
$$C = Enc_k(CTR) \oplus P \quad P = Enc_k(CTR) \oplus C$$

Global Counter (8B)

2

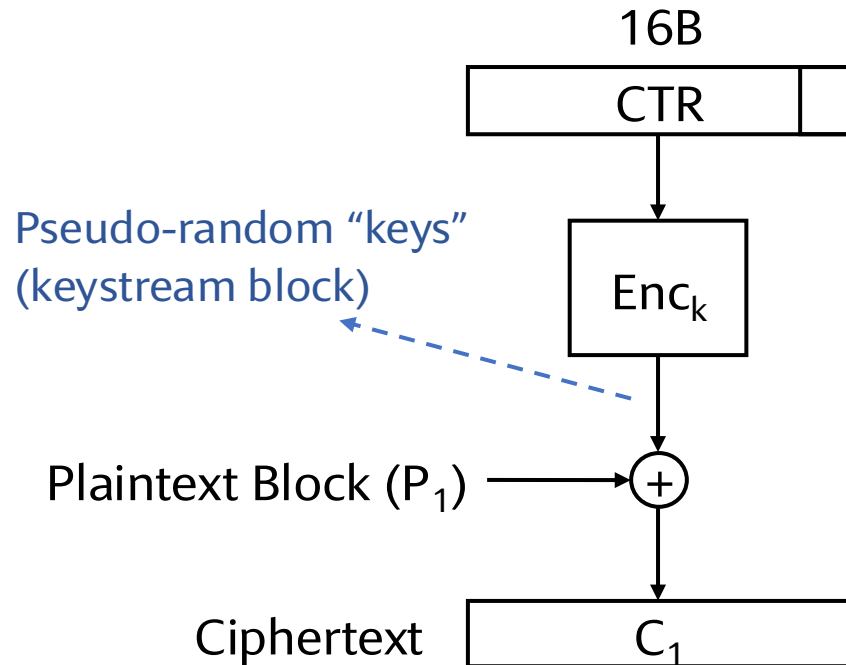


Evicted Cacheline



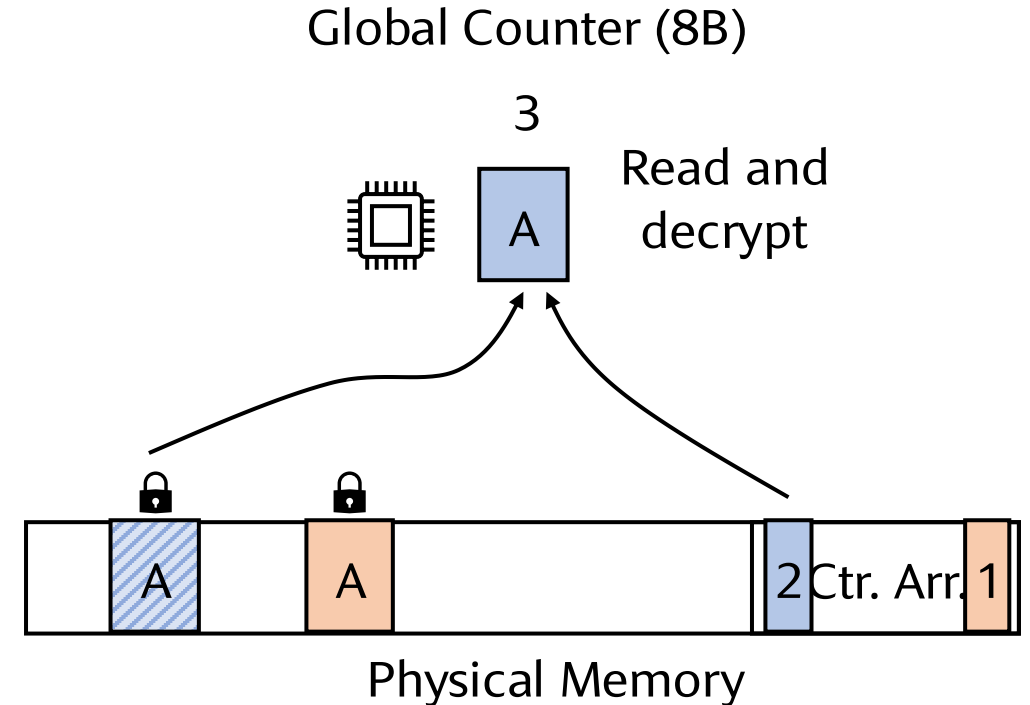
Physical Memory

Counter (CTR) Mode



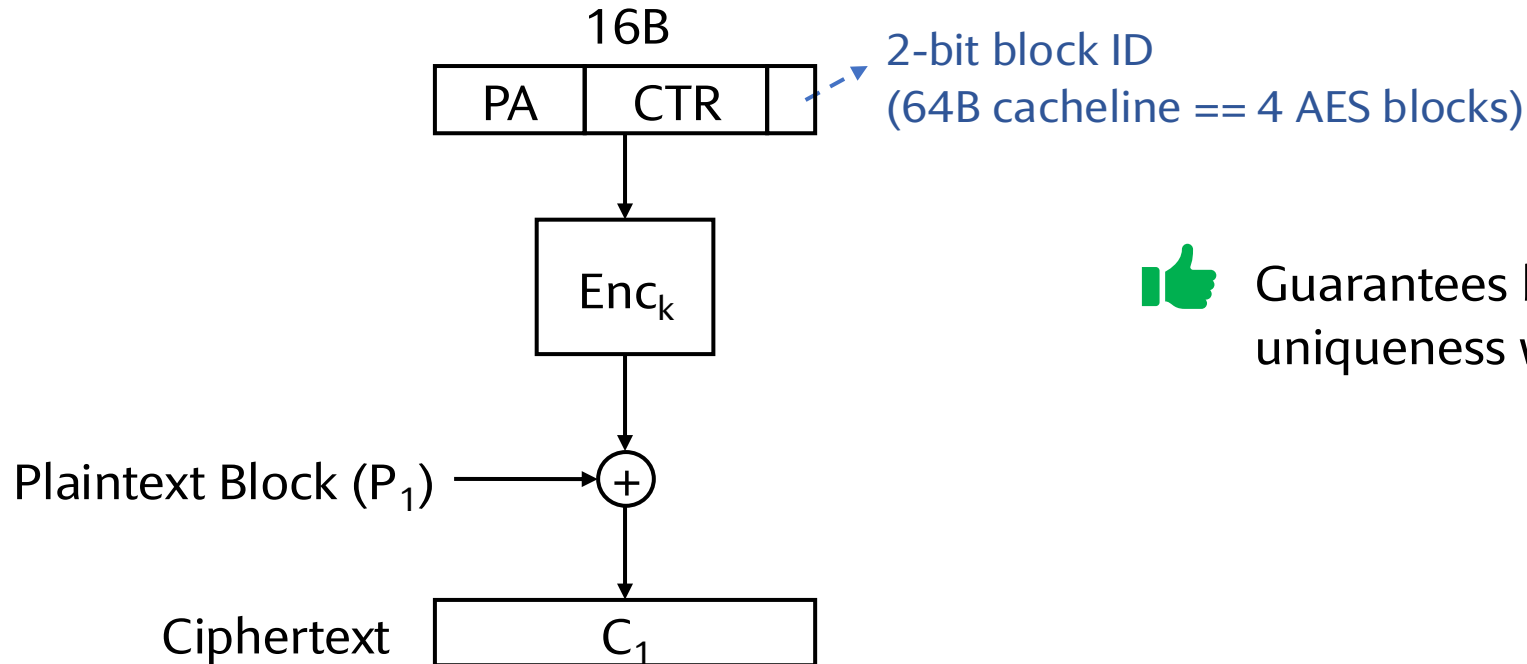
$$C = Enc_k(CTR) \oplus P \quad P = Enc_k(CTR) \oplus C$$

Ctr overflow? Re-encrypt with a new key. Expensive!



👎 Large storage overhead

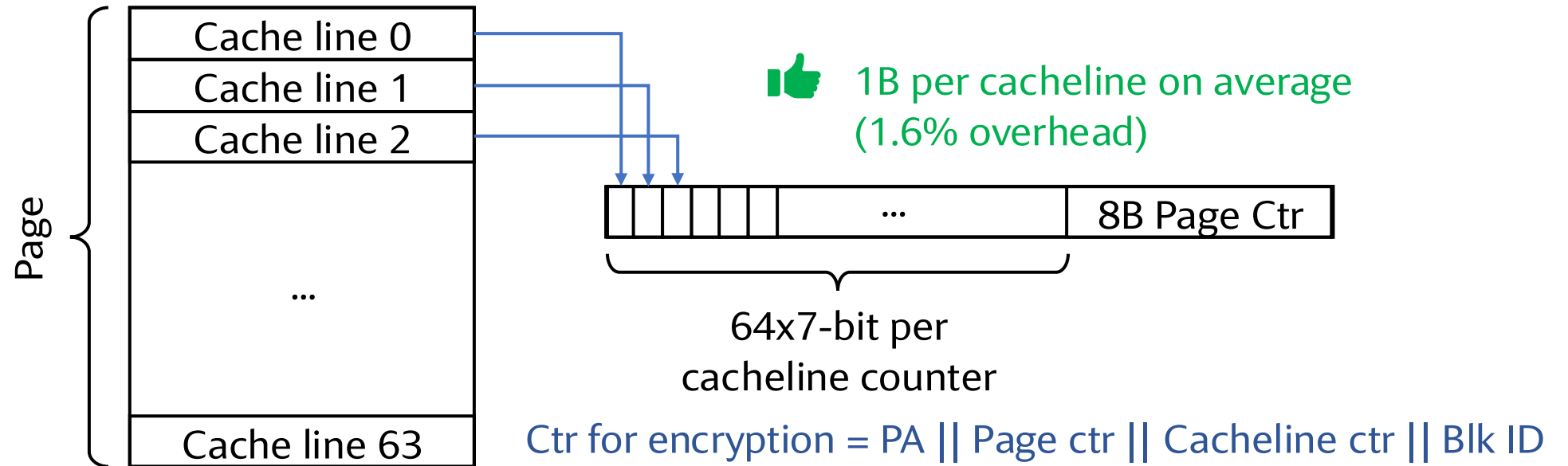
Smaller Cacheline Counter + Physical Address



Guarantees both spatial and temporal uniqueness while using a smaller counter

$$C = \text{Enc}_k(CTR) \oplus P \quad P = \text{Enc}_k(CTR) \oplus C$$

An Even More Compact Counter Scheme



Cacheline counter overflow? Increment the page counter, reset cacheline counters, re-encrypt the entire page

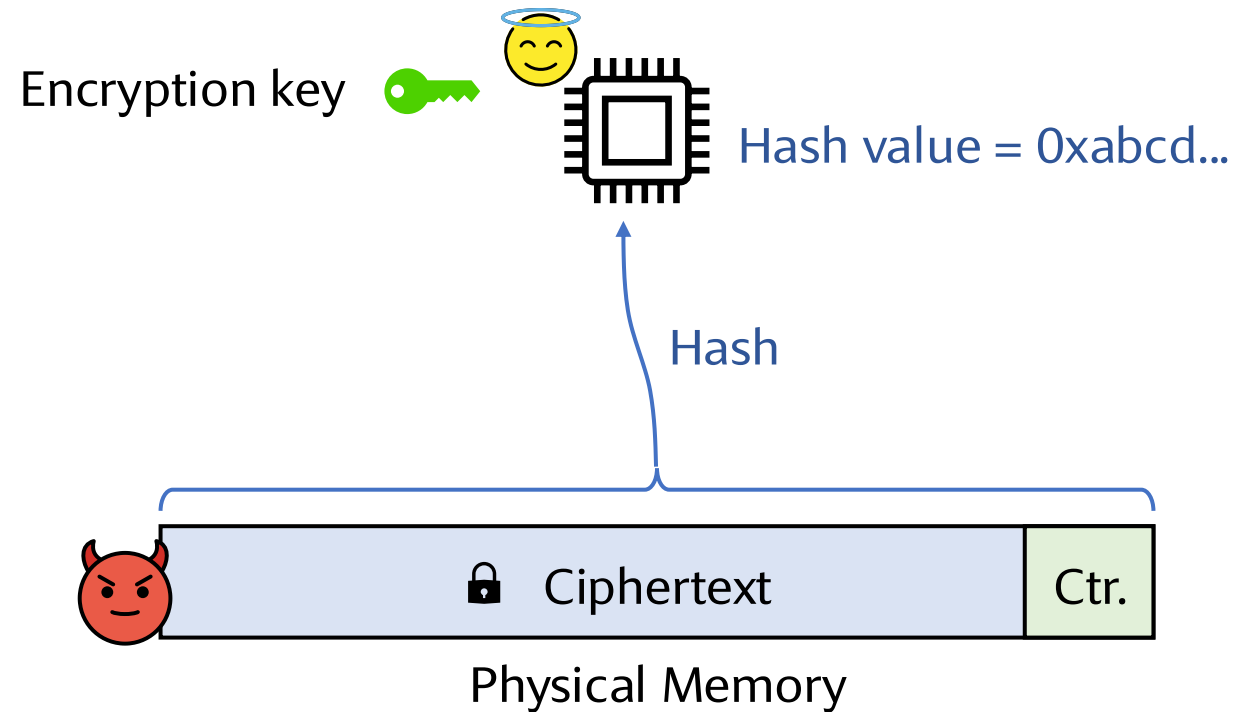
Integrity Protection Goals

- **No spoofing:** Attacker cannot replace the value
- **No splicing:** Attacker cannot exchange the value with another value from a different location
- **No replay:** Attacker cannot replay an old value from the same location

Use crypto hashes?

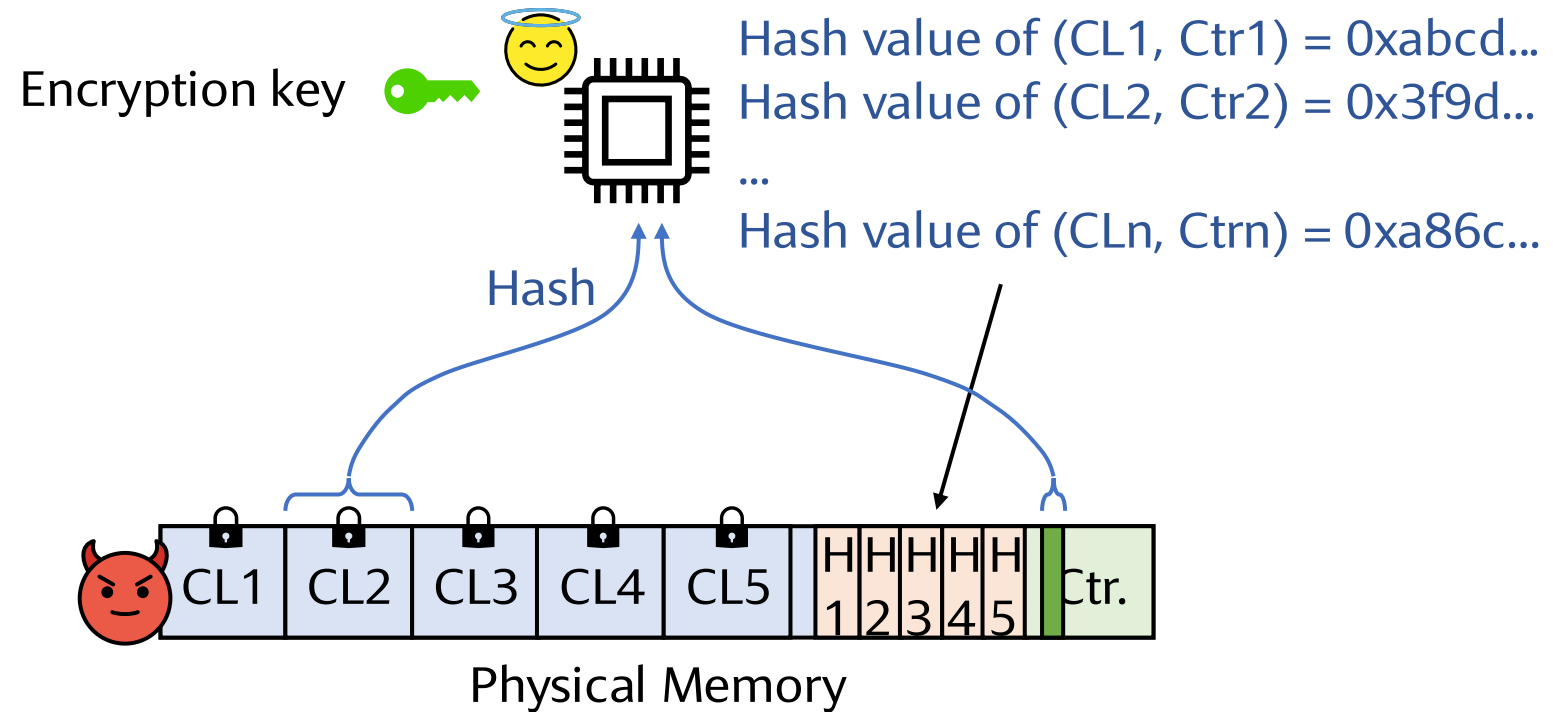
Naïve Memory Integrity Protection

Hash the entire memory and store the expected hash value on the chip



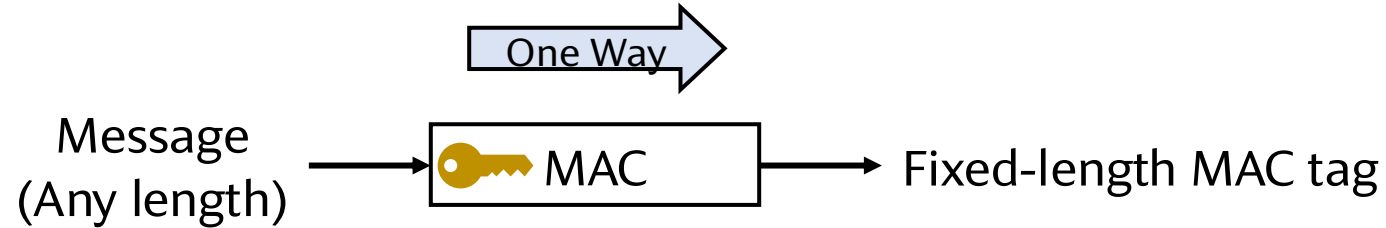
Naïve Memory Integrity Protection

Hash each individual cacheline and its counter



Not secure! Attacker can forge ciphertext blocks and their hashes

Hammer 5: Message Authentication Code (MAC)



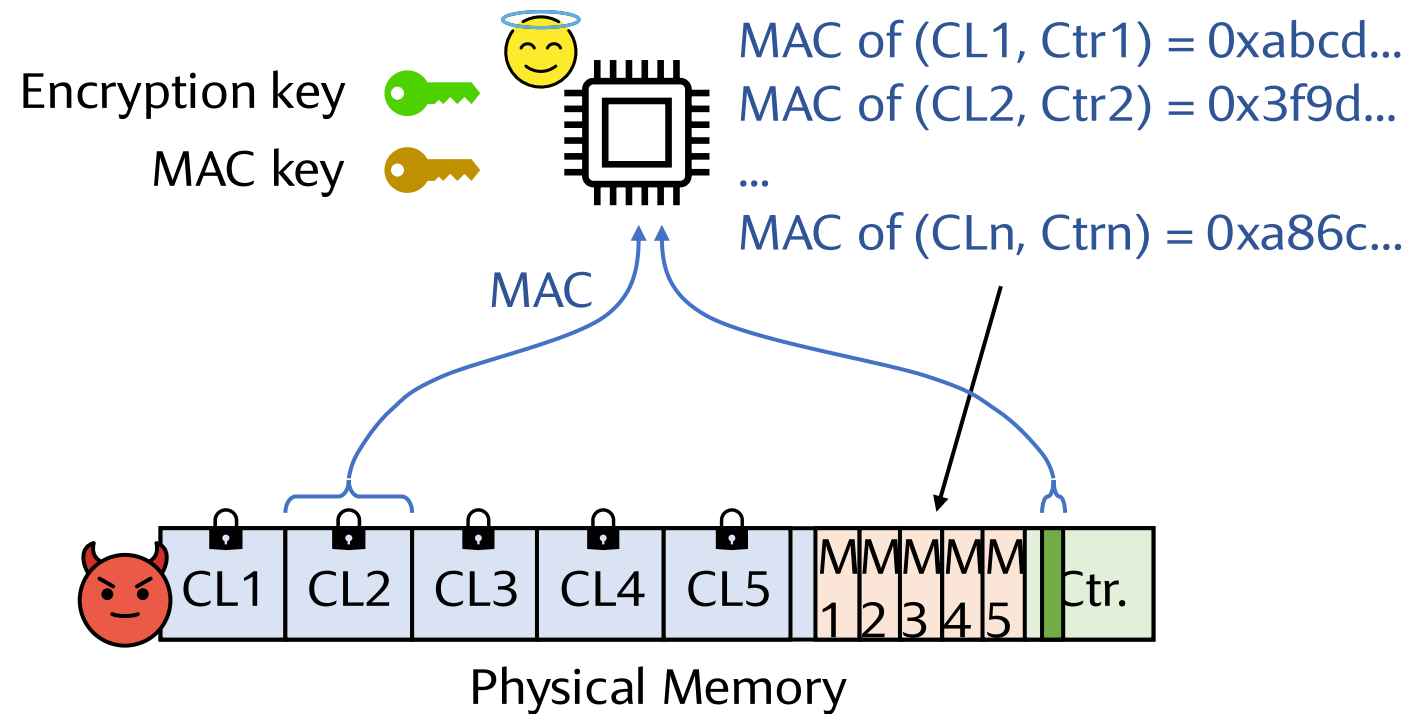
Properties:

- Verifier has the same key
- Only the person who has the key can produce the correct MAC tag
⇒ Correct MAC: The message is authentic

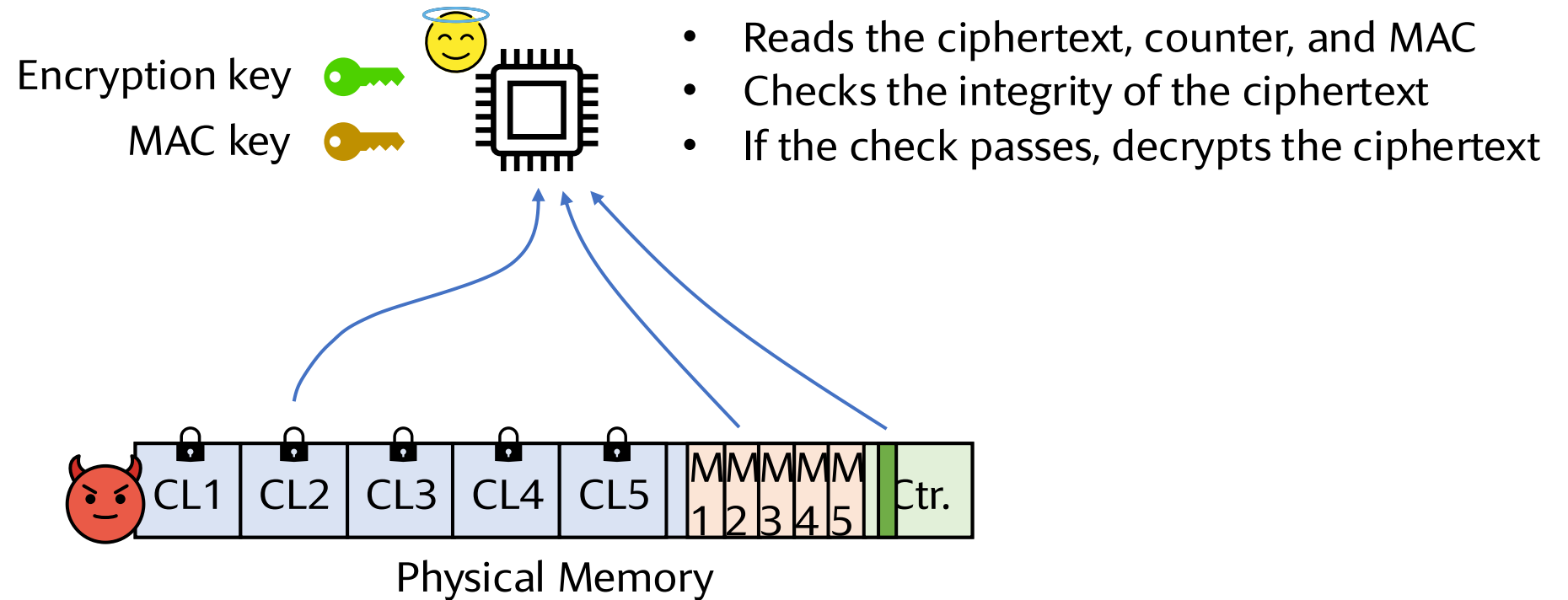
Examples:

- Hash-based MAC (HMAC): Turns a crypto hash function into a MAC construction (e.g., HMAC-SHA256)
- Poly1305: A dedicated MAC design by DJB. Commonly used with ChaCha20, a stream cipher

Memory Integrity Protection with MAC

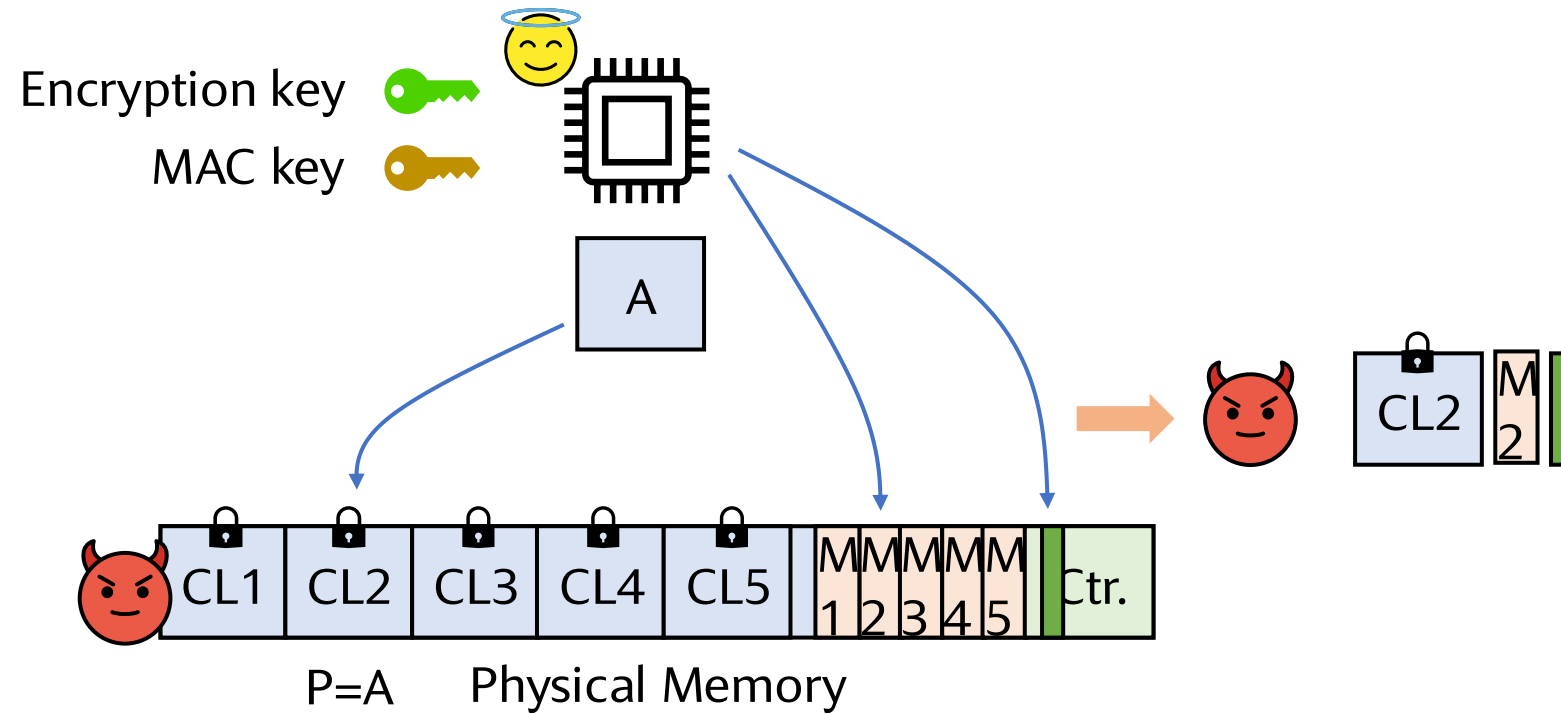


Accessing the Memory

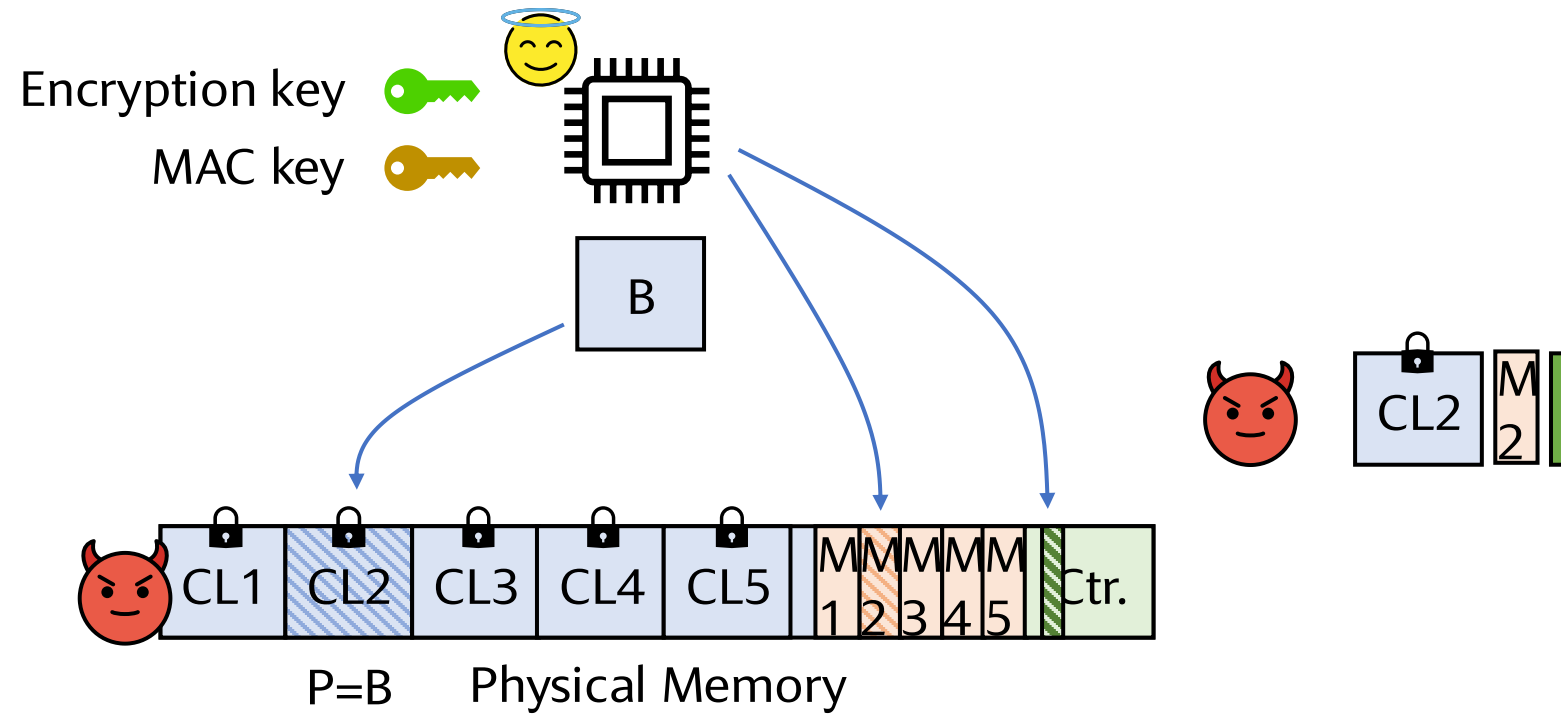


To prevent splicing, bind MAC to the PA---e.g., $\text{MAC Tag} = \text{MAC}_k(\text{Ciphertext}, \text{Ctr}, \text{PA})$

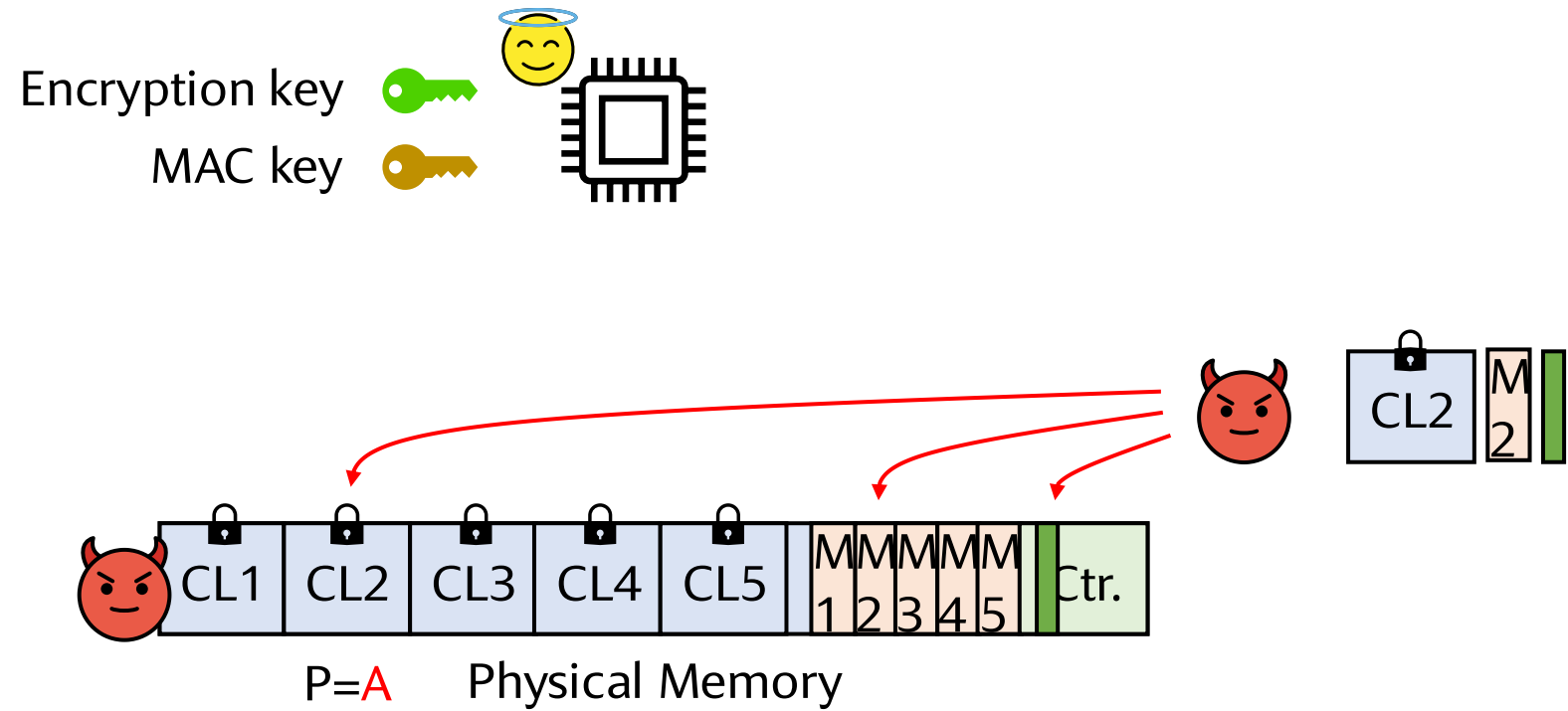
Wait, What About Freshness



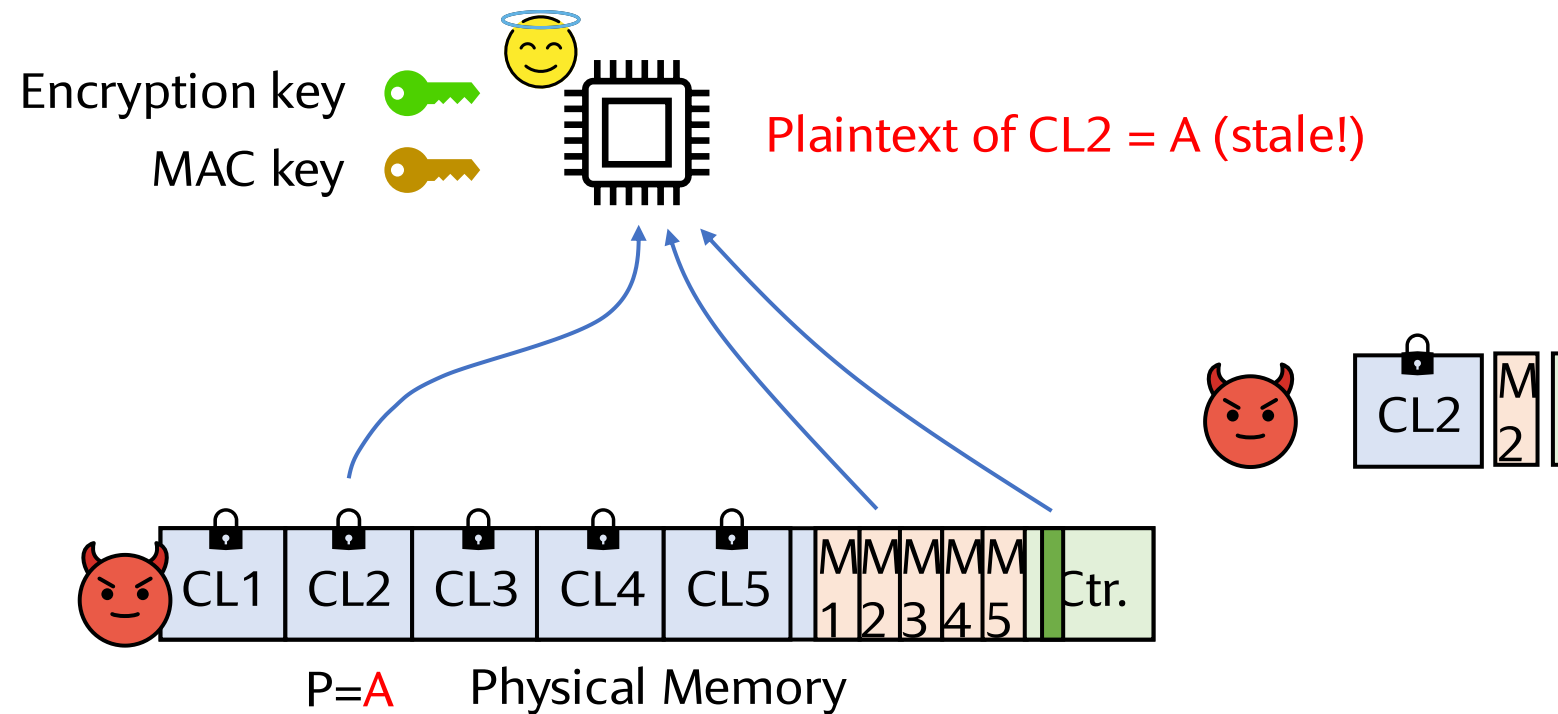
Wait, What About Freshness



Wait, What About Freshness

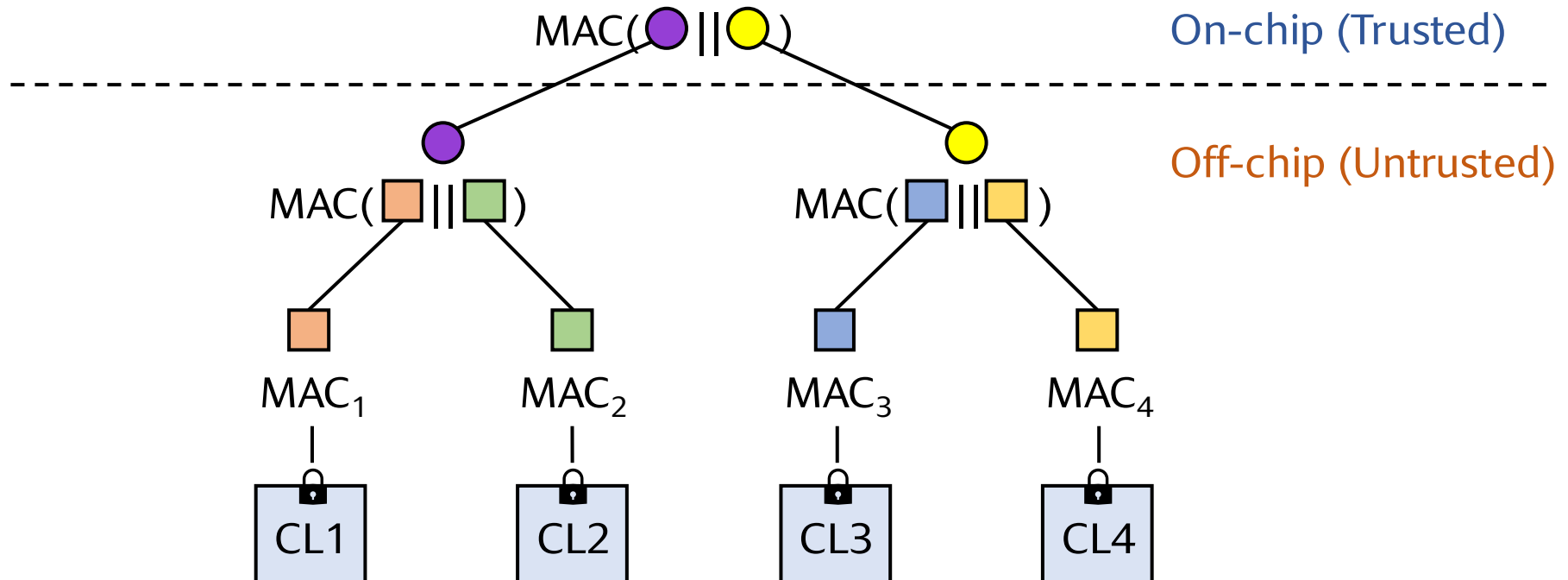
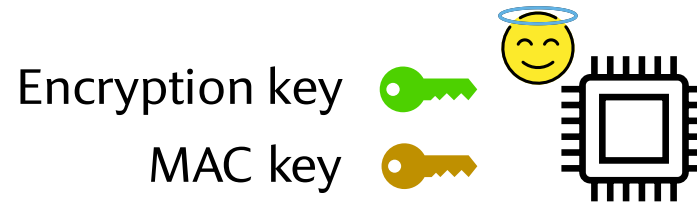


Wait, What About Freshness

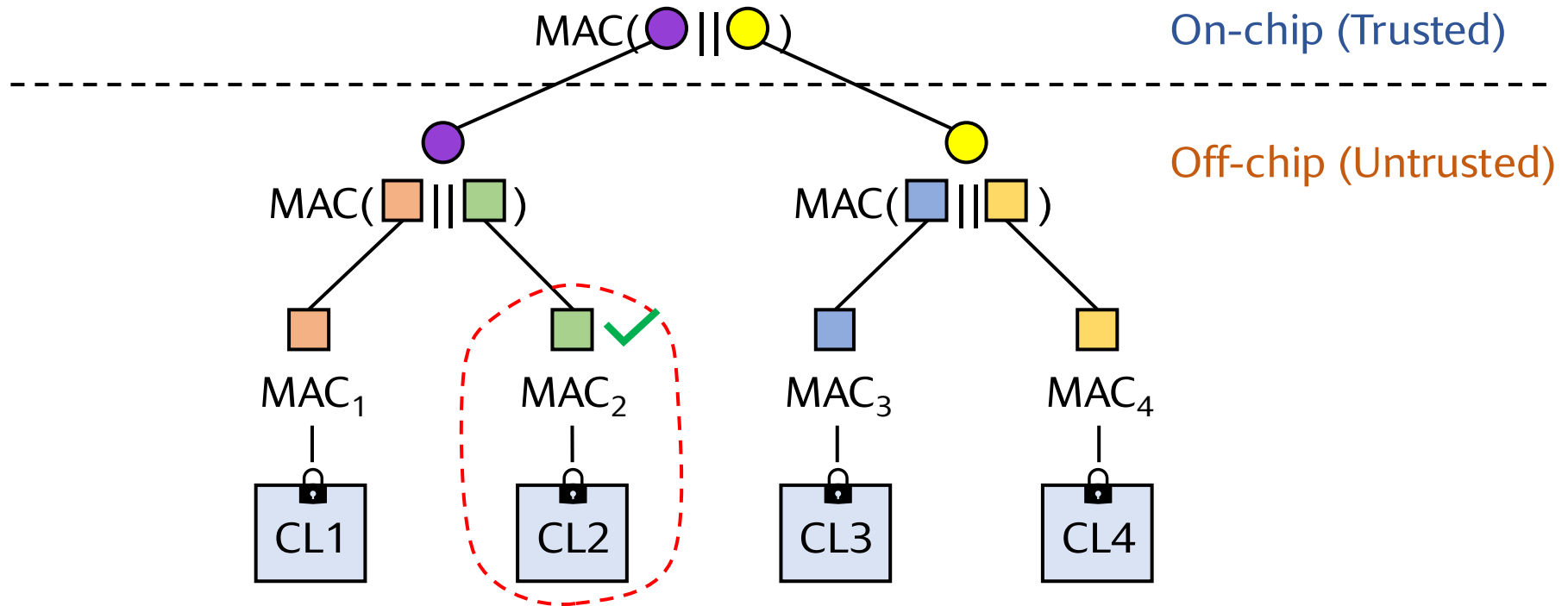
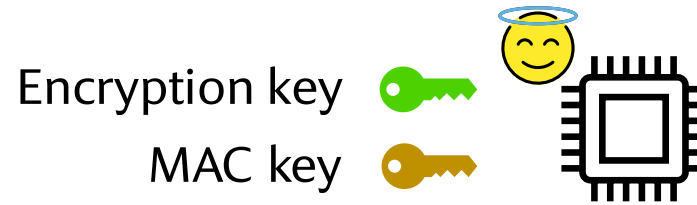


Authenticating a cache line in isolation isn't enough!

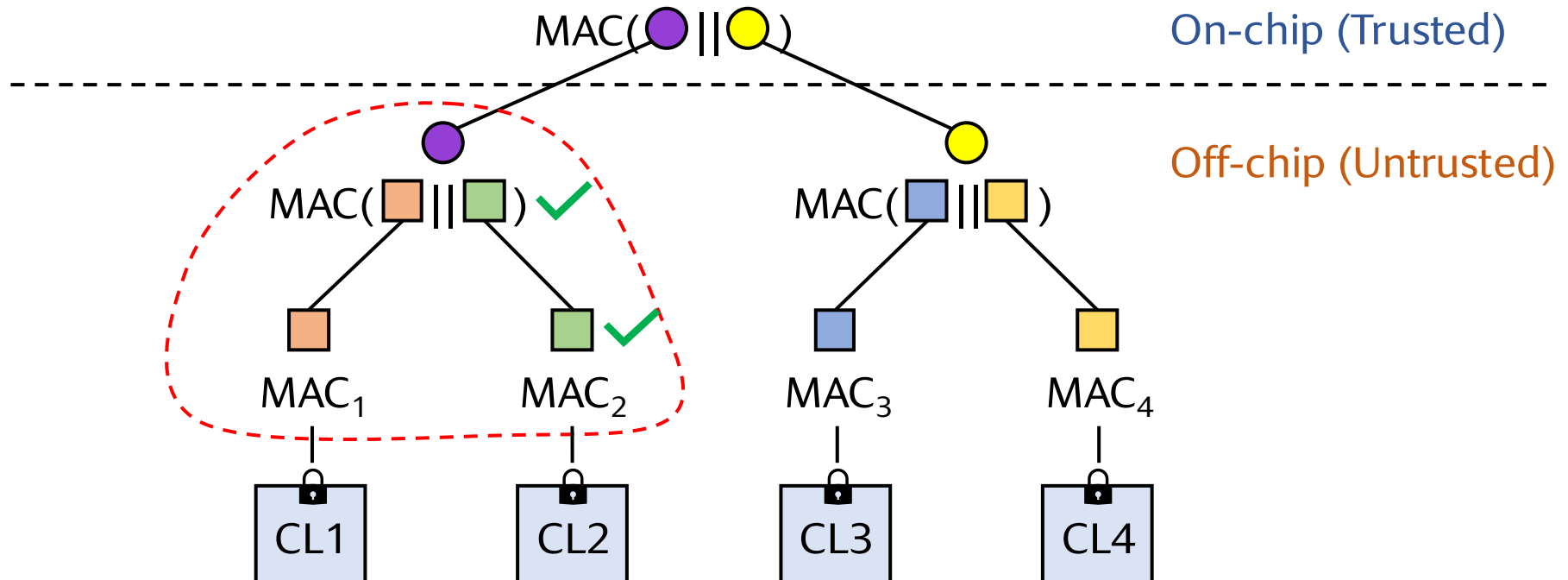
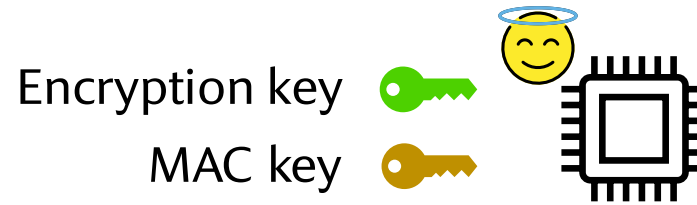
Hammer 6: Merkle (Integrity) Tree



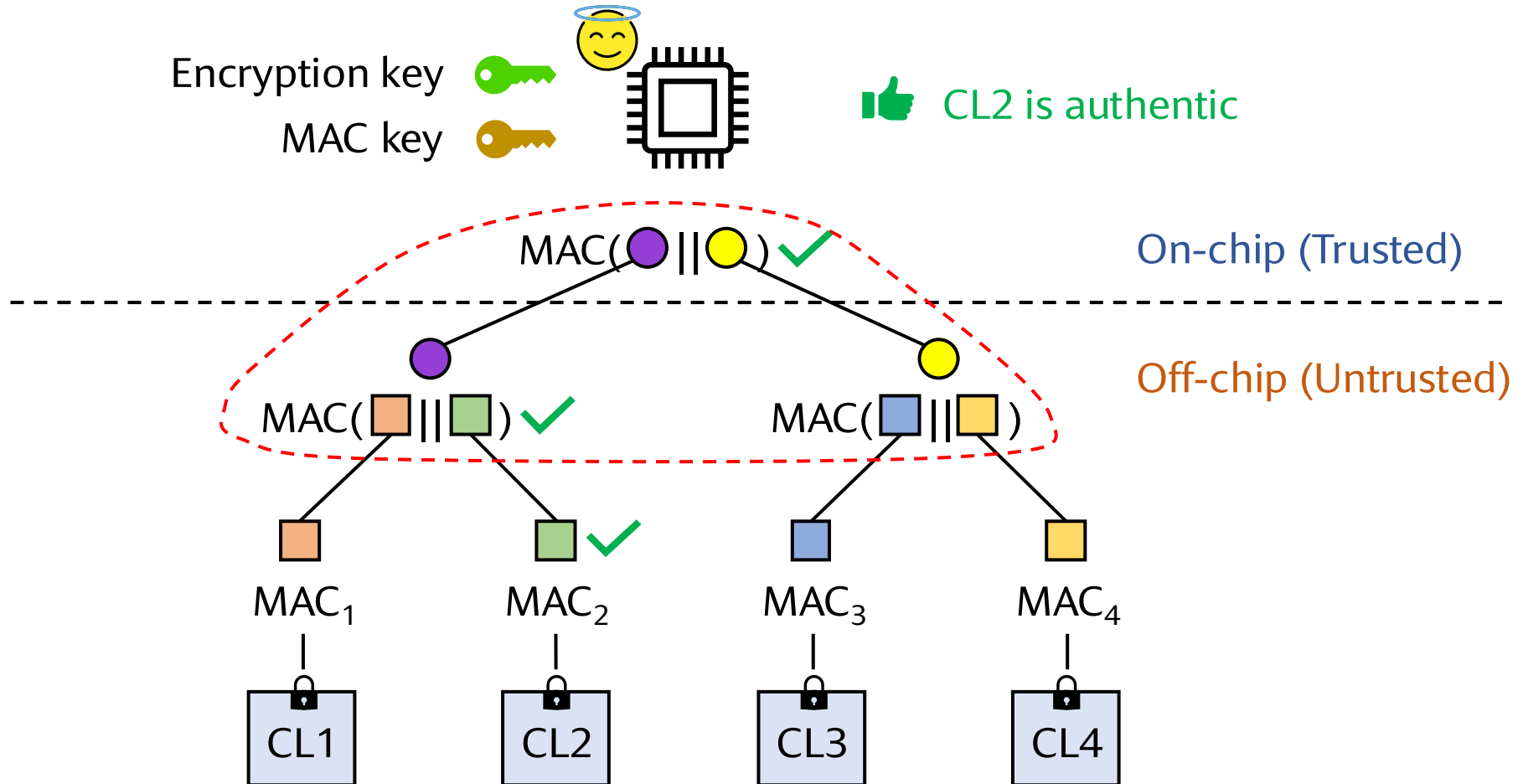
Verify the Integrity of C2



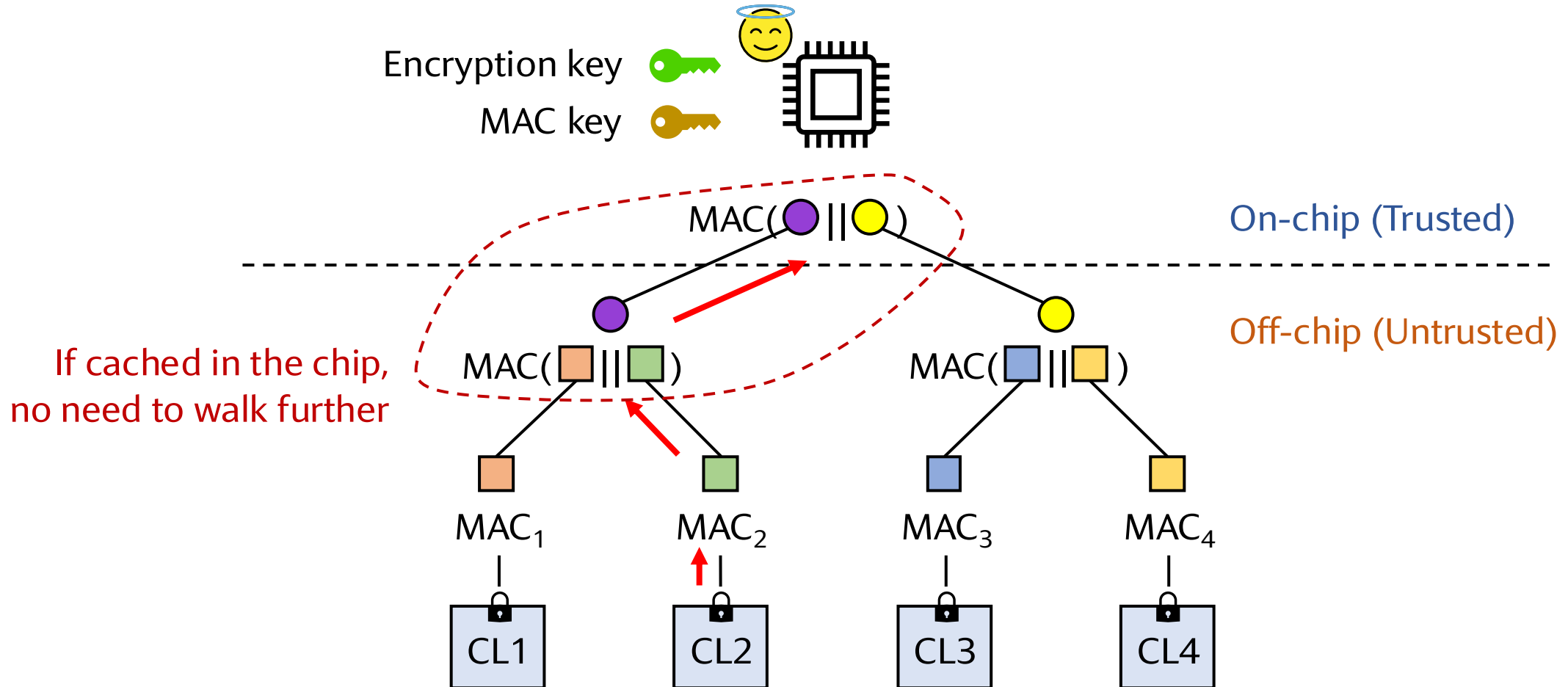
Verify the Integrity of C2



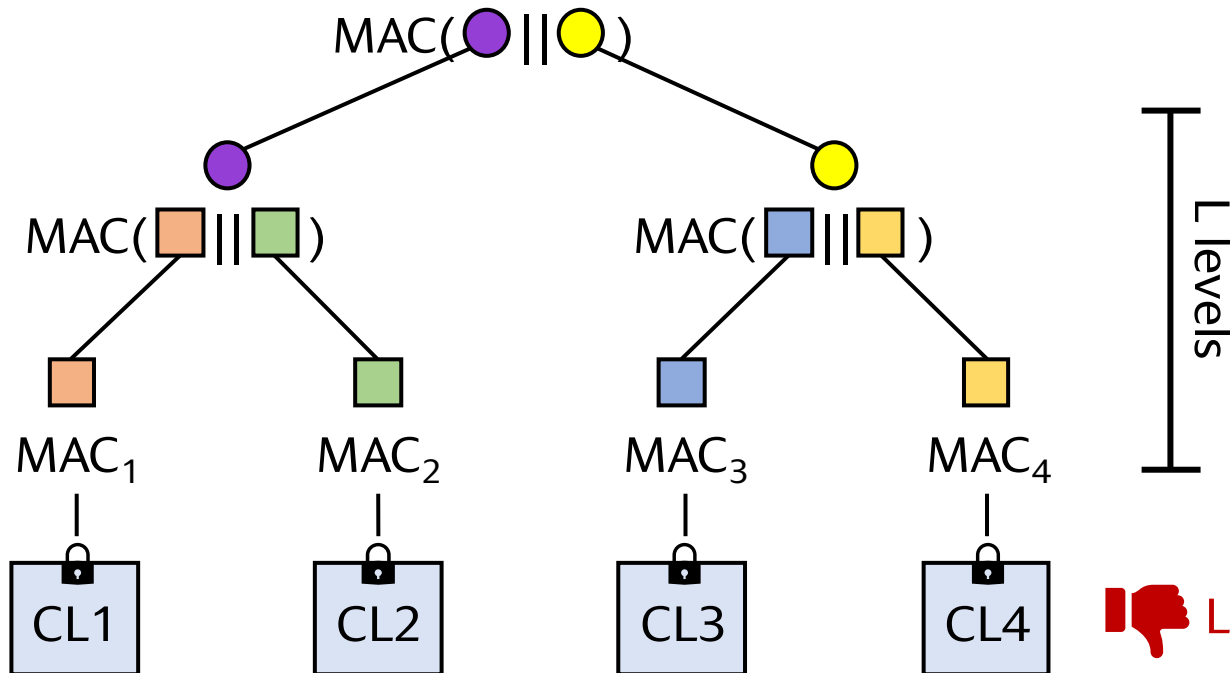
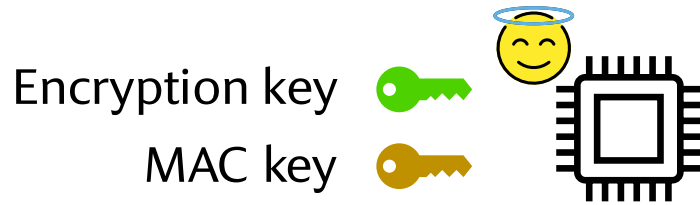
Verify the Integrity of CL2



Verify the Integrity of CL2



How Large is the Merkle Tree



Assume 64-bit (or 8-B) MAC

$\Rightarrow$ 8 MACs per cacheline $\Rightarrow$ Fan-out = 8

Total leaves # $N_{leaf} = 8^L$

Total tree nodes # $N_{node} = (8^{L+1} - 8)/7$

cachelines covered = N_{leaf}

Absolute storage cost = $8N_{node}$

Storage overhead =

$$\frac{8N_{node}}{64N_{leaf}} = \frac{8^{L+1} - 8}{7 \times 8^{L+1}} = \frac{1 - 8^{-L}}{7} \approx \frac{1}{7}$$

 Large storage overhead. Many memory accesses